APRESENTAÇÃO FINAL · ENTREGA 2

TUPÃ

Da ideia ao hardware.



Michel Lutegar · André Costa
Felipe Dutrain · Guilherme Lopes

LINGUAGENS FORMAIS E COMPILADORES · IBMEC

O PROBLEMA

O mesmo teste, em duas linguagens.

firmware/teste_hardware.ino — 51 linhas

```
void loop() {
  int b = digitalRead(BTN);
  int ir = digitalRead(IR_PIN);
  if (b ≠ lastBtn) {
    Serial.println(b == LOW ? "Botao: PRESSIONADO"
                            : "Botao: solto");
    lastBtn = b;
  }
  if (ir ≠ lastIr) {
    Serial.println(ir == LOW ? "IR: obstaculo detectado"
                             : "IR: livre");
    lastIr = ir;
  }
  digitalWrite(LED_R, b == LOW);
  if (rfid.PICC_IsNewCardPresent() &&
      rfid.PICC_ReadCardSerial()) {
    // ... UID, LED, PICC_HaltA()
  }
}
```

examples/teste_hardware.tupa — 23 linhas

```
# botão e IR já registram no Serial sozinhos
ao botao(1) {
    led.vermelho();
    esperar(300);
}

ao obstaculo {
}

ao cartao {
    escrever("Cartao UID: " + cartao.uid);
    led.verde();
    esperar(800);
}
```

A LACUNA

Ninguém une português, texto e hardware real.

Arduino IDE
C++ cru. O hardware aparece inteiro.

MakeCode, Blockly
Blocos. Não ensinam texto.

Flipper Zero
Ferramenta pronta. Não ensina a programar.

Portugol Studio
Português, mas sem placa.

Tupã
Texto em português que grava na placa.

A LINGUAGEM

Eventos em português.

examples/controle_nfc.tupa — 12 linhas

```
dispositivo controle {

    inteiro tentativas = 3;

    ao botao(1) {
        # le um cartao NFC
        cartao = nfc.ler();

        se cartao.valido == verdadeiro {
            led.verde();
            esperar(1500);
        } senao {
            led.vermelho();
        }
    }
}
```

`ao iniciar` `ao botao(1)` `ao obstaculo` `ao cartao`

Legível.
Orientada a evento.
Sem pino à vista.

ANÁLISE LÉXICA

Uma regex mestre, 44 tokens.

compilador/lexer.py — regex mestre

```
_MESTRE = re.compile("|".join(
    f"(?P<{nome}>{padrao})" for nome, padrao in _ESPECIFICACAO))
```

18 PALAVRAS RESERVADAS

dispositivo · ao · se · senao · repetir · esperar · escrever · nfc · led · ...

14 OPERADORES

\+ · - · * · / · = · == · ≠ · ≥ · && · || · ...

1 IDENTIFICADOR · 3 LITERAIS

TK_ID · TK_INT · TK_FLOAT · TK_STRING

7 DELIMITADORES · FIM DE ARQUIVO

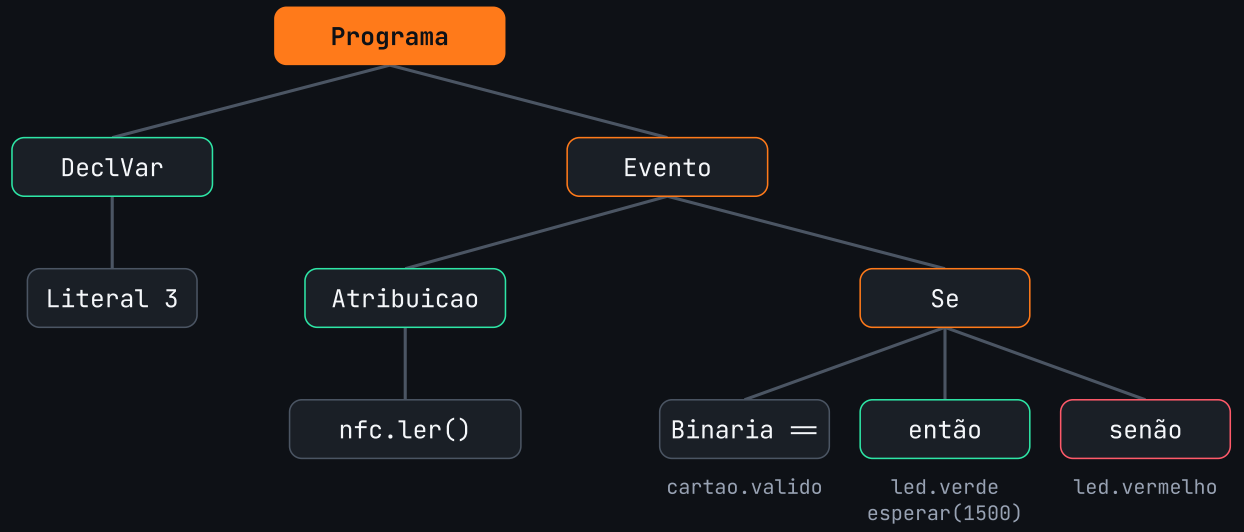
( ) · { } · , · . · ; · TK_EOF

ANÁLISE SINTÁTICA

Descendente recursivo, 17 tipos de nó.

EBNF resumida

```
programa ::= "dispositivo" ID
             "{" item* "}"
item     ::= evento | comando
evento   ::= "ao" "botao" "(" INT ")" bloco
           | "ao" ID bloco
bloco    ::= "{" comando* "}"
se       ::= "se" expr bloco
             ( "senao" bloco )?
expr     ::= ou
ou       ::= e ( "||" e )*
  ...  igualdade, termo, fator
posfixo  ::= primario
             ( "." ID | "(" args? ")" )*
```



ANÁLISE SEMÂNTICA

Erros que ensinam.

```
led.roxo();
```

```
erro semantico - linha 3, coluna 12
    metodo inexistente no modulo 'led'
    disponiveis: verde(), vermelho(), apagar()
    led.roxo()

compilacao interrompida: 1 erro
```

```
ir.enviar("x");
```

```
erro semantico - linha 3, coluna 11
    metodo inexistente no modulo 'ir'
    disponiveis: detectado()
    o 'ir' e um sensor de obstaculo (emissor e
    receptor no mesmo modulo): nao existe IR TX
    nem controle remoto
    ir.enviar()

compilacao interrompida: 1 erro
```

Saída real do compilador. Linha, coluna e o que existe de verdade.

GERAÇÃO DE CÓDIGO

Do arquivo da placa ao .ino.

placas/esp32_wrover.json — placa

```
"pinos": {
  "nfc_ss": 5,    "nfc_rst": 22,
  "led_verde": 25, "led_vermelho": 26,
  "ir": 34,        "botoes": { "1": 4 }
},
"nfc": {
  "timeout_ms": 3000,
  "uids_autorizados": [
    "33 7A 87 F7", "43 A3 6F F5"
  ]
}
```

controle_nfc_gerado.ino — setup() e loop()

```
#define DEBOUNCE_MS    50
#define NFC_TIMEOUT_MS 3000

// nfc.ler(): espera até 3 s pelo cartão
while (!lido && (millis() - inicio) < NFC_TIMEOUT_MS) {
  lido = nfcTentar(c);
  if (!lido) delay(20);
}

void loop() {
  if ((millis() - botao1_mudou) > DEBOUNCE_MS
      && leitura ≠ botao1_estavel) {
    botao1_estavel = leitura;  // um disparo por aperto
  }
}
```

pinagem · setup() / loop() · debounce 50 ms · espera de 3 s no NFC · lista de UIDs

MÉTRICAS

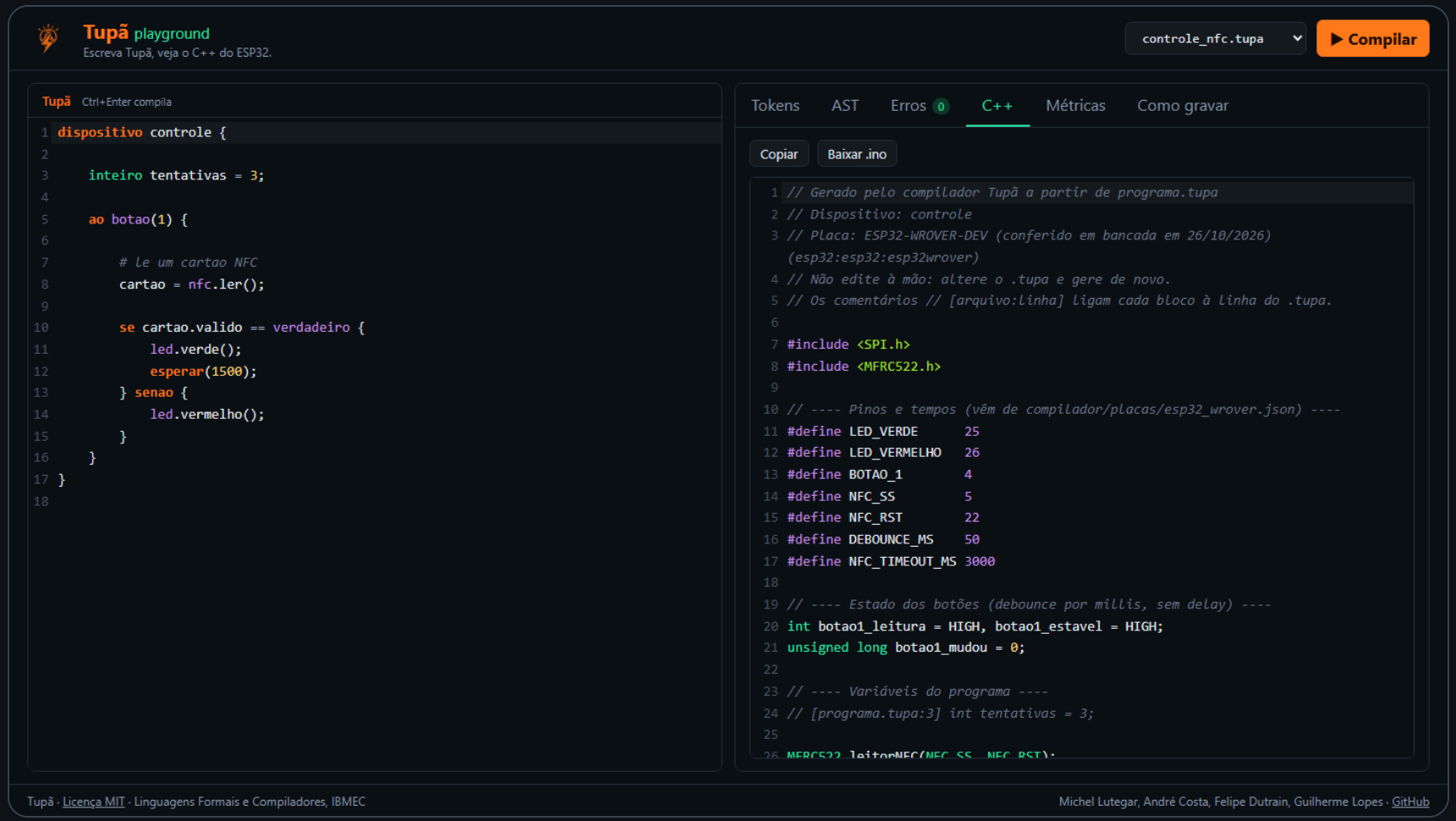
Menos código para dizer a mesma coisa.

EXEMPLO	TUPÃ	C++ GERADO	RAZÃO	MANUAL
controle_nfc	12	100	8,3x	não medido
teste_hardware	23	112	4,9x	51

Linhas de código, sem brancos e sem comentários · 50 testes passando

PLAYGROUND

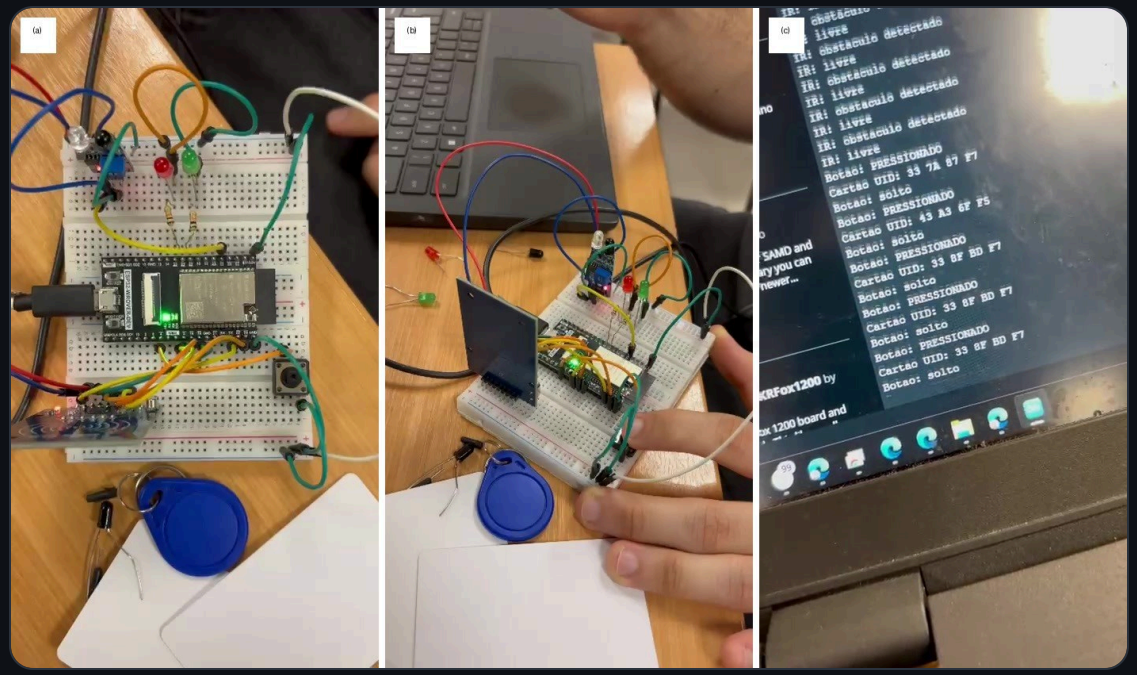
No ar, sem instalar nada.



tupa.mlutegar.com

HARDWARE E DEMO AO VIVO

Do site à placa.



1. Compilar no site
2. Baixar o .ino
3. Gravar na placa
4. Passar o cartão:

`33 7A 87 F7 · liberado · LED verde`

`33 8F BD F7 · negado · LED vermelho`

`sem cartão · negado após 3 s`

ESP32-WROVER-DEV · RC522 · sensor IR · botão · 2 LEDs · ≈ R$ 120

CONCLUSÃO

Entregue, no ar e aberto.

PRÓXIMOS PASSOS

- OTA
- IR TX
- testes com estudantes

CÓDIGO

- Licença MIT
- tupa.mlutegar.com

Perguntas? A placa está aqui do lado.